Early-Stopping Self-Consistency at a matched budget on a small open-weight model

1. Objective

To evaluate early-stopping self-consistency (ESC; window 5, at most 20 samples) for arithmetic word problems on a fixed random subset of 50 questions from the GSM8K test set (1,319 questions), using the open-weight Qwen2.5-1.5B-Instruct model run locally, compared against fixed-size self-consistency (SC) at a matched sample budget, measured by exact-match accuracy and by samples, tokens, sequential rounds and wall-clock latency per question, each reported as mean and standard deviation over three seeded runs.

The objective was implemented as approved, with no change of scope.

2. What was implemented

Baseline. Chain-of-thought with greedy decoding (one call) and SC (Wang et al., 2023) at every sample count k from 1 to 20.

Method under study. ESC (Li et al., 2024, Algorithm 1): samples are drawn in windows of 5; if all answers in a window are equal, sampling stops and that answer is returned; otherwise the next window is drawn, and after 20 samples the usual majority vote is taken.

One pool of samples for all methods. For each run and question the model is sampled in 4 sequential rounds of 5. SC-k reads the first k samples and ESC reads whole rounds until its rule fires. Model, prompt, decoding settings and samples are therefore identical across methods, and the only thing that differs is the stopping rule (src/methods.py, 90 lines).

Ablation and context. (i) Random stopping: ESC's per-question sample counts are shuffled across questions (500 shuffles), which keeps the budget and removes the signal. (ii) Window sizes 2, 3, 4, 10. (iii) Adaptive-Consistency with the Beta rule at 0.95 (Aggarwal et al., 2023), re-implemented from the paper.

Infrastructure. A resumable sampler that writes one JSON trace per run and question (prompt, every reply, tokens, per-round wall-clock time), retries failed calls and re-draws a round whose timing was spoilt by a stall; an evaluator that rebuilds every table from the traces without calling the model; and demo.py, which runs ESC live on one question.

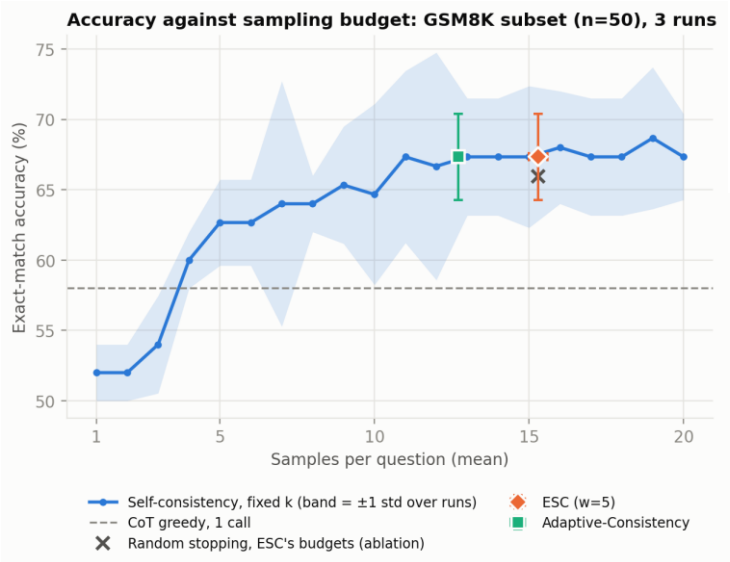
3. Experimental setup (pinned configuration)

Model	Qwen2.5-1.5B-Instruct, GGUF q4_k_m (sha256 6a1a2eb6...434e9407e)
Runtime	llama.cpp llama-server b11429, CPU only, 4 threads, 5 slots
Decoding	temperature 0.7, top-p 0.9, top-k off, repeat penalty 1.0, max 256 new tokens; greedy baseline at temperature 0
Prompt	system message + 3 worked examples of my own + question (prompts/, about 420 prompt tokens)
Tasks	GSM8K test, subset seed 465, 50 questions (data/subset_ids.json)
Runs	3, seeds 11, 22, 33; sample seed = run x 10^6 + question x 100 + index
Checker	exact match on the number after "Final answer", else the last number in the reply; no LLM judge
Date, cost	6 October 2026; 3000 sampled generations, 0.28M completion tokens, 3.1 h of CPU, zero rupees

4. Results

Method	Accuracy %	Samples	Rounds	Prompt tok	Compl. tok	Total tok	Latency s
CoT greedy (1 call)	58.0	1	1	420	90	510	6.5
SC-5	62.7 ± 3.1	5.00	1.00	420	466	885	19.8 ± 2.4
SC-10	64.7 ± 6.4	10.00	1.00	420	918	1338	38.4 ± 3.7
SC-15 (matched, floor)	67.3 ± 5.0	15.00	1.00	420	1376	1795	56.6 ± 4.9
SC-16 (matched, ceil)	68.0 ± 4.0	16.00	1.00	420	1467	1886	60.2 ± 4.8
SC-20 (full budget)	67.3 ± 3.1	20.00	1.00	420	1836	2256	74.6 ± 4.7
ESC (w=5, L=20)	**67.3 ± 3.1**	**15.30**	**3.06**	**1295**	**1487**	**2782**	**60.7 ± 5.3**
Adaptive-Consistency	67.3 ± 3.1	12.71	12.71	5391	1257	6648	52.0 ± 5.9
Random stopping (ablation)	65.9 ± 3.4	15.30	3.06	1295			

Table 1. Mean ± standard deviation over 3 runs of 50 questions; costs are per question. Prompt tokens: one prompt per round. Regenerate with python src/evaluate.py.



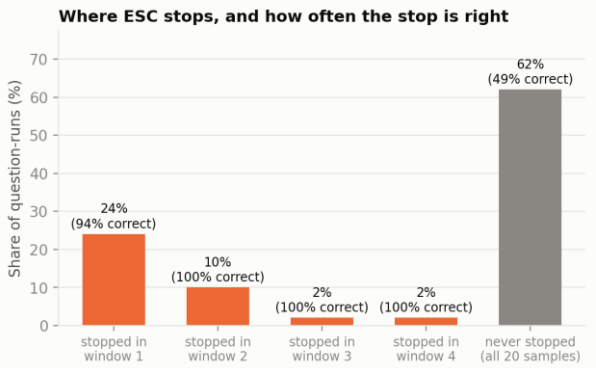


Figure 1 (left). Accuracy against samples per question. Figure 2 (right). Where ESC stops and how often the stop is correct.

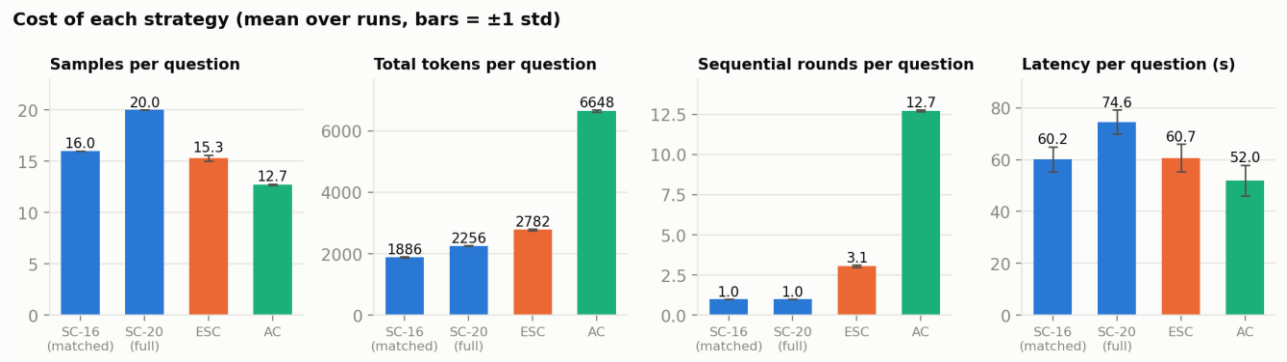


Figure 3. Cost per question of the matched and full SC baselines, ESC and Adaptive-Consistency.

5. Analysis

Baseline against published numbers. Greedy chain of thought scores 58.0% here; the Qwen team reports 73.2% on GSM8K for this model. I did not close that gap. The likely causes are the 4-bit quantised weights, my own three-example prompt with replies kept to a few sentences (about 92 tokens), and the size of the subset: at n = 50 one question is 2.0 points. For ESC itself, Table 8 of the paper (L = 20) gives 10.14 samples for GPT-3.5-Turbo (SC accuracy 85%) and 17.06 for Llama-2 7b (21%). This model, with SC accuracy 67%, uses 15.30, which is consistent with the published pattern that the saving follows model accuracy. SC also behaves as published: accuracy rises from 52.0% at k = 1 to 67.3% at k = 20.

Finding 1: no detectable accuracy gain at a matched budget. ESC scores 67.3 ± 3.1% using 15.30 samples per question. SC given the same budget scores 67.3 ± 5.0% (k = 15) and 68.0 ± 4.0% (k = 16). The paired difference is +0.0 ± 2.0 points against the floor and -0.7 ± 1.2 against the ceiling; in both cases the spread across runs is larger than the difference, so neither can be told apart from zero. The paper's claim of higher accuracy at equal cost (0.2 to 0.9 points on GSM8K, no variance given) is not something an experiment of this size can confirm or refute.

Finding 2: the saving in samples is modest, and in tokens it is negative. ESC stops early on 38% of question-runs and saves 23% of the samples of SC-20, with identical answers (agreement 100%). But each window is a new request that carries the 420-token prompt again, so ESC sends 1295 prompt tokens per question against 420 for SC, and its total is 2782 tokens against 2256: 23% more than the method it is meant to make cheaper. It also needs 3.1 sequential rounds where SC needs one. On this CPU the measured latency follows the sample count (61 s against 75 s), because local generation is compute-bound; behind a parallel API the round count would be the latency. Adaptive-Consistency stops sooner (12.7 samples) at the same accuracy, but as one request per sample it sends 5391 prompt tokens.

Finding 3 (ablation): the stopping signal is real but worth little here. With ESC's budgets handed out at random, accuracy is 65.9%, so the unanimity signal is worth +1.4 ± 0.6 points. 2 of the 57 early stops landed on a wrong answer (Figure 2; results/failure_cases.md), the confident wrong stop that Gap 4 of Phase 1 pointed at. The signal is precise, but for this model it fires on fewer than half of the question-runs. Window size trades cost against accuracy (Figure 4): w = 2 uses 6.7 samples for 62.0 ± 2.0%, w = 10 uses 18.8 for 67.3 ± 3.1%.

Failure case read from a trace (traces/run_11/q0099.json). Mary receives 18 new plants, already has 2 on each of 40 ledges, and gives away 1 per ledge. Gold answer 58. In run 11 the first five samples all answer 40, so ESC stops after one window and returns 40; the same happens in run 33. The trace shows the cause: every reply in the window computes 2 * 40 = 80 and 80 - 40 = 40 and never uses the 18 new plants from the first sentence. This is the confident wrong stop of Gap 4: unanimity measures how consistent the model is, not whether it is right. Here the stop did no harm, since 18 of the 20 samples say 40 and SC-20 returns 40 as well, so ESC lost nothing and saved 15 samples. The opposite pattern is in q1052: 13 of 20 samples repeat the slip 140 - 50 + 100 = 290, the seven dissenters block every window, and ESC spends the full budget on a question where more samples cannot help. 12 of the 50 questions are wrong in every run (results/failure_cases.md).

Checker sensitivity, and what it does to Finding 1. 13.5% of replies omit the "Final answer" line. The main table reads the last number of such a reply; the strict reading counts it as no answer (results/strict/). Under the strict reading ESC scores 67.3 ± 4.2% at 16.70 samples and is ahead of matched SC by +3.3 ± 1.2 points (k = 16) and +2.7 ± 1.2 (k = 17). Findings 2 and 3 are the same under both readings; Finding 1 is not. With three runs of 50 questions, where one question is 2 points, I do not regard a matched-budget gain as either shown or ruled out: it appears under one scoring rule and disappears under the other.

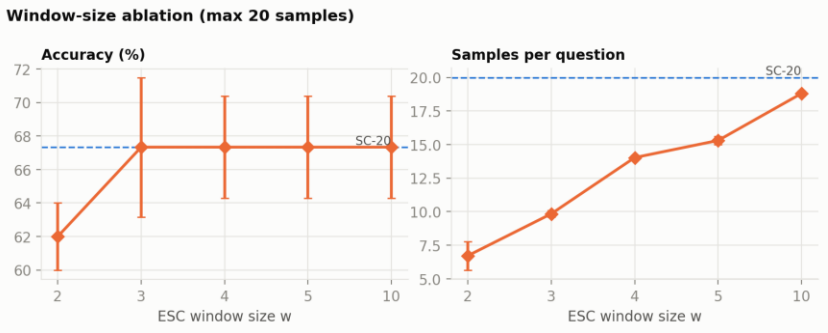


Figure 4. Window-size ablation.

6. Limitations, and what four more weeks would add

- Small n: 50 questions, one model, one prompt, one benchmark. Standard deviations of 3 to 3 points mean only large effects are visible; the conclusion is that no gain was detected, not that none exists.
- L = 20, not the paper's 40; a larger budget gives ESC more room to save.
- The three runs share one pool per run across methods. This makes the comparison paired, but SC-k always reads the first k samples.
- Latency was measured on a shared laptop and varied with its load; the round count is the more portable figure. SC-k latency for k not a multiple of 5 is prorated within a round.
- The checker was revised once, after the first traces showed missing answer lines; both readings are reported. The 40-sample audit sheet (results/checker_audit.csv) is provided for hand checking.

With four more weeks: (1) the full 1,319 questions and L = 40 on a GPU, to reach the paper's resolution of under one point; (2) a second, stronger open-weight model (Qwen2.5-7B) to trace the saving against model accuracy; (3) a real latency study against a hosted API with parallel sampling; (4) a token-aware ESC that keeps the prompt cached across windows, which would remove the overhead found in Finding 2.

An AI assistant (Claude) was used to write and debug the code and to draft this report; the numbers come from the traces in this submission.